Some formula of fast algorithm

Zhengbang Zhou

PACS numbers:

I. THEORY OF THE ISDF ALGORITHM

The Interpolative Separable Density Fitting (ISDF) algorithm aims at reducing the evaluation time of the FFT applied to wavefunctions products. Let's indeed introduce a set of real–space point distributed in the direct lattice unit cell of a solid: $\{\mathbf{r}_i\}$, with $i = 1, \ldots, N_r$.

In addition to this large set of point we define a subset $\{\mathbf{x}_\alpha\}$, with $\alpha = 1, \ldots, N_\alpha$. We now define

$$\rho_{n\mathbf{k}m\mathbf{p}}(\mathbf{r}) = \overline{u_{n\mathbf{k}}(\mathbf{r})}\nu_{m\mathbf{p}}(\mathbf{r})\,\mathbf{e}^{\mathbf{i}\mathbf{G}_0\cdot\mathbf{r}}. \tag{1}$$

Here, both $u_{n\mathbf{k}}$ and $\nu_{m\mathbf{p}}$ are Bloch wavefunctions, $\mathbf{k}$-points $\mathbf{k}$, $\mathbf{p}$, and reciprocal lattice $\mathbf{G}_0$ satisfies

$$\mathbf{k} - \mathbf{q} + \mathbf{G}_0 = \mathbf{p} \tag{2}$$

for some $\mathbf{q}$ in the first Brillouin zone. We use different letters to indicate that they belong to different sets, where $n \in \mathcal{N}$, $m \in \mathcal{N}'$, $\mathbf{k} \in \mathcal{K}$, and $\mathbf{p} \in \mathcal{K}'$. The reason to do so will be specifically discussed in the implementation section.

We now evaluate Eq. (1) on the $\{\mathbf{r}_i\}$ points and introduce some *helper functions* $p_\alpha(\mathbf{r}_i)$

$$\begin{aligned}\rho_{n\mathbf{k},m\mathbf{p}}(\mathbf{r}_i) &= \sum_\alpha p_\alpha(\mathbf{r}_i)\,\overline{u_{n\mathbf{k}}(\mathbf{x}_\alpha)}u_{m\mathbf{p}}(\mathbf{x}_\alpha)\,\mathbf{e}^{\mathbf{i}\mathbf{G}_0\cdot\mathbf{x}_\alpha} \\ &= \sum_\alpha p_\alpha(\mathbf{r}_i)\,\rho_{n\mathbf{k}m\mathbf{p}}(\mathbf{x}_\alpha).\end{aligned} \tag{3}$$

The number of helper functions is determined by the user-defined parameter `ISDF_real` (see 1) as

$$N_\alpha = \texttt{ISDF\_real} \cdot \sqrt{|\mathcal{N}||\mathcal{N}'||\mathcal{K}||\mathcal{K}'|}. \tag{4}$$

We now introduce

$$\underline{\mathcal{M}}_{i,n\mathbf{k}m\mathbf{p}} = \rho_{n\mathbf{k}m\mathbf{p}}(\mathbf{r}_i), \tag{5a}$$

$$\underline{\mathcal{P}}_{i,\alpha} = p_\alpha(\mathbf{r}_i), \tag{5b}$$

$$\underline{\mathcal{C}}_{\alpha,n\mathbf{k}m\mathbf{p}} = \overline{u_{n\mathbf{k}}(\mathbf{x}_\alpha)}u_{m\mathbf{p}}(\mathbf{x}_\alpha)\,\mathbf{e}^{\mathbf{i}\mathbf{G}_0\cdot\mathbf{x}_\alpha}. \tag{5c}$$

Thanks to Eq. (5) we can rewrite Eq. (3) as

$$\underline{\mathcal{M}} = \underline{\mathcal{P}}\,\underline{\mathcal{C}}. \tag{6}$$

We need to generate the subset $\{\mathbf{x}_\alpha\}$ and the helper functions explicitly. We use the *Centroidal Voronoi Tessellation* procedure (CVT) to select the subset $\{\mathbf{x}_\alpha\}$. In the concept of CVT, every point $\mathbf{r}_i$ has a weight w_i, and in our problem it is based on the densities and one possible expression is

$$w_i = \sum_{n\mathbf{k}} \rho_{n\mathbf{k}n\mathbf{k}}(\mathbf{r}_i). \tag{7}$$

CVT aims to partition the set $\{\mathbf{r}_i\}$ into a number of disjoint *clusters* $\{\mathbf{C}_\alpha\}$. Each cluster $\mathbf{C}_\alpha$ has a center $\mathbf{c}_\alpha$, and is represents by a *generator* $\mathbf{x}_\alpha \in \mathbf{C}_\alpha$. The clusters, its center, and its generator are interdependent through the following equations

$$\mathbf{C}_\alpha = \{\mathbf{r}_i \mid \mathrm{dist}(\mathbf{r}_i, \mathbf{x}_\alpha) \le \mathrm{dist}(\mathbf{r}_i, \mathbf{x}_\beta) \text{ for all } \alpha \ne \beta\}, \tag{8}$$

$$\mathbf{c}_\alpha = \frac{\sum_{\mathbf{r}_i \in \mathbf{C}_\alpha} w_i \mathbf{r}_i}{\sum_{\mathbf{r}_i \in \mathbf{C}_\alpha} w_i}, \tag{9}$$

$$\mathbf{x}_\alpha = \{\mathbf{r}_i \in \mathbf{C}_\alpha \mid \mathrm{dist}(\mathbf{r}_i, \mathbf{c}_\alpha) \le \mathrm{dist}(\mathbf{r}_j - \mathbf{c}_\alpha) \text{ for all } i \ne j\}. \tag{10}$$

In practice, we iteratively use Eq. (8), Eq. (9), and Eq. (10) to update the clusters, its centers, and its generators until the convergency reached. To provide a practical convergence criterion, we introduce a loss function f

$$f(\{\mathbf{r}_i\},\{\mathbf{C}_\alpha\}) = \sum_\alpha \sum_{i\in\mathbf{C}_\alpha} \|\mathbf{r}_i - \mathbf{x}_\alpha\|_2^2, \tag{11}$$

and the convergence criterion is defined based on it.

Once the subset $\{\mathbf{x}_\alpha\}$ are decided, we solve the helper functions using least-square, which is to find the helper function $p_\alpha(\mathbf{r}_i)$ by minimizing the following summation

$$\left\{ \sum_i \sum_{n\in\mathcal{N},\mathbf{k}\in\mathcal{K}} \sum_{m\in\mathcal{N}',\mathbf{p}\in\mathcal{K}'} \left| \sum_\alpha \left(\overline{u_{n\mathbf{k}}(\mathbf{x}_\alpha)} u_{n'\mathbf{k}'}(\mathbf{x}_\alpha) p_\alpha(\mathbf{r}_i) \mathrm{e}^{\mathrm{i}\mathbf{G}_0\mathbf{x}_\alpha} \right) - \overline{u_{n\mathbf{k}}(\mathbf{r})} u_{n'\mathbf{k}'}(\mathbf{r}_i) \mathrm{e}^{\mathrm{i}\mathbf{G}_0\mathbf{r}} \right|^2 \right\}. \tag{12}$$

Thanks to Eq. (6) we have a concise way to express the helper functions as

$$\underline{\mathcal{P}} = \left(\underline{\mathcal{M}}\,\underline{\mathcal{C}}^+\right)\left(\underline{\mathcal{C}}\,\underline{\mathcal{C}}^+\right)^{-1}. \tag{13}$$

Here the $^+$ refers to the conjugate transpose of a matrix.

To calculate (13), we have to calculate $\underline{\mathcal{C}}\,\underline{\mathcal{C}}^+$ and $\underline{\mathcal{M}\mathcal{C}}^+$. Notice $\underline{\mathcal{C}}$ is a submatrix of $\underline{\mathcal{M}}$ by taking some rows, thus $\underline{\mathcal{C}}\,\underline{\mathcal{C}}^+$ is a submatrix of $\underline{\mathcal{M}\mathcal{C}}^+$. We only calculate $\underline{\mathcal{C}}\,\underline{\mathcal{C}}^+$.

We can express the element of $\underline{\mathcal{M}}\,\underline{\mathcal{C}}^+$ as

$$(\underline{\mathcal{M}}\,\underline{\mathcal{C}}^+)_{i,\alpha} = \sum_{n\in\mathcal{N},\mathbf{k}\in\mathcal{K}} \sum_{m\in\mathcal{N}',\mathbf{p}\in\mathcal{K}'} \overline{u_{n\mathbf{k}}(\mathbf{r}_i)} \nu_{m\mathbf{p}}(\mathbf{r}_i) u_{n\mathbf{k}}(\mathbf{x}_\alpha) \overline{\nu_{m\mathbf{p}}(\mathbf{x}_\alpha)} \mathrm{e}^{\mathrm{i}\mathbf{G}_0\mathbf{r}_i} \mathrm{e}^{-\mathrm{i}\mathbf{G}_0\mathbf{x}_\alpha} \tag{14}$$

$$= \sum_{\mathbf{k}\in\mathcal{K},\mathbf{p}\in\mathcal{K}'} \mathrm{e}^{\mathrm{i}\mathbf{G}_0(\mathbf{r}_i-\mathbf{x}_\alpha)} \Big(\sum_{n\in\mathcal{N}} \overline{u_{n\mathbf{k}}(\mathbf{r}_i)} u_{n\mathbf{k}}(\mathbf{x}_\alpha) \Big) \Big(\sum_{m\in\mathcal{N}'} \nu_{m\mathbf{p}}(\mathbf{r}_i) \overline{\nu_{m\mathbf{p}}(\mathbf{x}_\alpha)} \Big) \tag{15}$$

We introduce two sets of $\mathbf{k}$-dependent matrices

$$\underline{\Lambda}^{\mathbf{k}}_{i,\alpha} = \sum_{n\in\mathcal{N}} \overline{u_{n\mathbf{k}}(\mathbf{r}_i)} \nu_{n\mathbf{k}}(\mathbf{x}_\alpha), \qquad \underline{\Gamma}^{\mathbf{p}}_{i,\alpha} = \sum_{m\in\mathcal{N}'} \overline{\nu_{m\mathbf{p}}(\mathbf{r}_i)} \nu_{m\mathbf{p}}(\mathbf{x}_\alpha), \tag{16}$$

and Eq. (14) is represented into a summation over $\mathbf{k}$-indice

$$(\underline{\mathcal{M}}\,\underline{\mathcal{C}}^+)_{i,\alpha} = \sum_{\mathbf{k}\in\mathcal{K}} \sum_{\mathbf{p}\in\mathcal{K}'} \mathrm{e}^{\mathrm{i}\mathbf{G}_0(\mathbf{r}_i-\mathbf{x}_\alpha)} \underline{\Lambda}^{\mathbf{k}}_{i,\alpha} \overline{\underline{\Gamma}^{\mathbf{p}}_{i,\alpha}} \tag{17}$$

Once we have $\underline{\mathcal{M}}\,\underline{\mathcal{C}}^+$, we can obtain $\underline{\mathcal{C}}\,\underline{\mathcal{C}}^+$ by taking corresponding rows.

In practical applications, we need the density in the truncated reciprocal-space grid $\mathcal{G}$. To do so, we need the value of helper functions on reciprocal grid $\mathbf{G}$, and we reduce overall time cost of obtaining $p_\alpha(\mathbf{G})$ by first perform FFT on matrix $\underline{\mathcal{M}\mathcal{C}}^+$ since

$$\begin{aligned} p_\alpha(\mathbf{G}) &= \frac{\mathrm{vol}}{N_\mathrm{r}} \sum_i \sum_\beta \mathrm{e}^{\mathrm{i}\mathbf{G}\mathbf{r}_i} (\underline{\mathcal{M}\mathcal{C}}^+)_{i,\beta} (\underline{\mathcal{C}\mathcal{C}}^+)^{-1}_{\beta,\alpha} \\ &= \sum_\beta \Big(\sum_i \frac{\mathrm{vol}}{N_\mathrm{r}} \mathrm{e}^{\mathrm{i}\mathbf{G}\mathbf{r}_i} (\underline{\mathcal{M}\mathcal{C}}^+)_{i,\beta} \Big) (\underline{\mathcal{C}\mathcal{C}}^+)^{-1}_{\beta,\alpha}. \end{aligned} \tag{18}$$

Then, we express the density as

$$\begin{aligned} \rho_{n\mathbf{k}m\mathbf{p}}(\mathbf{G}) &= \int_\Omega \mathrm{d}\mathbf{r}\, \rho_{n\mathbf{k}m\mathbf{p}}(\mathbf{r}) \mathrm{e}^{\mathrm{i}\mathbf{G}\mathbf{r}} \\ &= \frac{\mathrm{vol}}{N_\mathrm{r}} \sum_i \rho_{n\mathbf{k}m\mathbf{p}}(\mathbf{r}_i) \mathrm{e}^{\mathrm{i}\mathbf{G}\mathbf{r}_i} \\ &= \frac{\mathrm{vol}}{N_\mathrm{r}} \sum_i \sum_\alpha \mathrm{e}^{\mathrm{i}\mathbf{G}_0\cdot\mathbf{x}_\alpha} \mathrm{e}^{\mathrm{i}\mathbf{G}\mathbf{r}_i} p_\alpha(\mathbf{r}_i) \overline{u_{n\mathbf{k}}(\mathbf{x}_\alpha)} \nu_{m\mathbf{p}}(\mathbf{x}_\alpha) \\ &= \sum_\alpha \mathrm{e}^{\mathrm{i}\mathbf{G}_0\cdot\mathbf{x}_\alpha} \Big(\frac{\mathrm{vol}}{N_\mathrm{r}} \sum_i p_\alpha(\mathbf{r}_i) \mathrm{e}^{\mathrm{i}\mathbf{G}\mathbf{r}_i} \Big) \overline{u_{n\mathbf{k}}(\mathbf{x}_\alpha)} \nu_{m\mathbf{p}}(\mathbf{x}_\alpha) \\ &= \sum_\alpha \mathrm{e}^{\mathrm{i}\mathbf{G}_0\cdot\mathbf{x}_\alpha} p_\alpha(\mathbf{G}) \overline{u_{n\mathbf{k}}(\mathbf{x}_\alpha)} \nu_{m\mathbf{p}}(\mathbf{x}_\alpha). \end{aligned} \tag{19}$$

We implement Eq. (7) to Eq. (11) in `isdf_get_xalpha`, Eq. (13) to Eq. (18) in `isdf_get_helper`, and Eq. (19) in `isdf_get_rho`. Descriptions of codes are provided in 4(a)i, 4(a)ii, and 4b, respectively.

II. IMPLEMENTATION OF THE ISDF ALGORITHM

A. ISDF Module: `mod_isdf`

To implement the ISDF algorithm in `Fortran`, we define a set of module-level variables and derived types within the module `isdf_m`. Since the physical information of a given system is unique, while multiple ISDF procedures may be applied to the same system, the system-related data are stored in module-level variables, ensuring they are shared across all procedures. In contrast, ISDF-specific data are encapsulated within derived types, allowing each ISDF procedure to maintain its own instance independently. This design enables multiple ISDF procedures to be performed on the same system without interference, with each procedure corresponding to a separate instance of the relevant derived type.

1. **Module-level Variables**:
   (a) `isdf_Nfftgrid`: Number of real-space grids.
   (b) `isdf_ngrho`: Number of reciprocal-space grids.
   (c) `isdf_Nbands`: Number of bands in `isdf_wf`.
   (d) `isdf_Nkpoints`: Number of **k**-points in `isdf_wf`.
   (e) `isdf_fftdim`: FFT dimension.
   (f) `isdf_wf`: Wavefunctions on real-space grids, size `isdf_Nfftgrid`×`isdf_Nbands`×`isdf_Nkpoints`.
   (g) `isdf_rset`: Real-space grids, in lattice coordinate.
   (h) `isdf_G0set`: $\mathbf{G}_0$ in Eq. (1), in lattice coordinate.
   (i) `fisttime`: A flag indicating whether material information needs to be read. The default value is `0`. When the program is called for the first time, it will read the material information and then set this value to `1`.
   (j) Some others ...
2. **Derived Types**:
   (a) `rho_fast`: Type used to store the density-related information.
   After constructed, user calls `rho_fast` **only** to get approximated density.
      i. `isdf_Nhelper`: Number of helper functions p_α in Eq. (3).
      ii. `ind_xalpha`: indices of $\{x_\alpha\}$ with respect to $\{\mathbf{r}_i\}$.
      iii. `helper_wf_g`: helper functions on truncated reciprocal space $\{p_\alpha(\mathbf{G})\}_{\|\mathbf{G}\|<\text{Ecut}}$ in Eq. (18).
   (b) `isdf_param`: Inputs and parameters to generate `rho_fast`.
   Users first set **necessary** properties in `isdf_param`, then call the **driver** to perform the isdf algorithm. A usage example is provided in the II C, illustrating the required parameters that must be set. Contains the following properties:
      i. `ib_start_end`: Set $\mathcal{N}$, $\mathcal{N}$ =`ib_start_end(1)`, ..., `ib_start_end(2)`.
      ii. `ob_start_end`: Set $\mathcal{N}$, $\mathcal{N}'$ =`ob_start_end(1)`, ..., `ob_start_end(2)`.
      iii. `ikbz_list`: Indices of $\mathcal{K}$.
      iv. `okbz_list`: Indices of $\mathcal{K}'$.
      v. `Nikbz`, `Nokbz`: Number of elements in $\mathcal{K}$ and $\mathcal{K}'$, respectively.
      vi. `isdf_Nhelper`: Number of helper functions p_α in Eq. (3).
      vii. `weightmethod`: Indicates how to generate CVT weight.
      viii. `init`: Indicates how to choose initial points.
      ix. `maxiter`, `filter`, `losstol`: parameters control breakout of main loop.
   (c) `isdf_rho_ws`: Workspace, use Eq. (19) to generate density in reciprocal space.
      i. `ib`, `ikbz`, `ob_start_end`, `okbz`, `is_ibz`: indices of density
      ii. `rho_approx`: space to save density on reciprocal grid, i.e.,

$$\texttt{rho\_approx(i, ob)} = \rho_{n_{\texttt{ib}}\mathbf{k}_{\texttt{ikbz}}n_{\texttt{ob}}\mathbf{k}_{\texttt{okbz}}}(\underline{R}_{\texttt{is\_ibz}}\mathbf{G}_i). \tag{20}$$

B. Other Modules and subroutines

All modules and subroutines, other than `mod_isdf`, are summarized below, along with a brief explanation of their roles.

1. **I/O**:
   (a) `isdf_INIT_activate.f90`, `isdf_INIT_load.f90`: Read inputs.
   (b) Output? What kind of output information we prefer?
2. **Memory**: `isdf_alloc.f90`, `isdf_dealloc.f90`
3. **Interface** with `Yambo`:
   (a) `isdf_yambo_driver.f90`: subroutine that uses to construct `rho_fast` in `yambo`.
      i. `isdf_read_yambo.f90`: subroutine that read module-level variables (material information) from `yambo`.
4. **ISDF Implementation**:
   (a) `isdf_main.f90` Main loop, prepare `rho_fast`, which can be used to construct approximated density.
      i. `isdf_get_xalpha`: Get $\mathbf{x}_\alpha$ in Eq. (3).
      ii. `isdf_get_helper`: Get $p_\alpha(\mathbf{G})$ in Eq. (18).
   (b) `isdf_get_rho.f90`: Get the approximated density $\rho(\mathbf{G})$ using `rho_fast` and given indices, see 2c.
5. **Utilize**: `mod_isdf_util.f90`
6. **Debug**: `mod_isdf_debug.f90`

C. Simplist usage

The ISDF module introduces the following input parameters. Users can specify these parameters in the input file to configure the ISDF procedure accordingly.

1. `DEBUG_int`: Debug information printing control. default `0`.
2. `ISDF_int`: ISDF algorithm control, use ISDF algorithm if `1`. default `0`.
3. `ISDF_real`: ISDF algorithm coefficient k, control the number of helper functions. default `8.0`.

The following code provides a minimal example of how to call the ISDF module. For computational effeciency, users are recommended to set `ob_start_end` (see 2c) as large as possible.

```
type(isdf_param) :: param ! isdf parameters
type(rho_fast)   :: rhof ! information to approximate density
type(isdf_rho_ws):: rhows! workspace for approximated density

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
! Calculate rhof
! Set param here, user should provide the set N, N', K, K'
! Here we takes the set to be full set as examples
param%ib_start_end = (/wf%b(1), wf%b(2)/)
param%ob_start_end = (/wf%b(1), wf%b(2)/)
param%ikbz_list = (/1, ..., k%nibz /)
param%okbz_list = (/1, ..., k%nbz /)

! Subroutine isdf_yambo_driver calculate rhof based on param.
call isdf_yambo_driver(E, k, q, param, rhof)
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
! call isdf_get_rho to get rho_approx in rhows%rho_approx
rhows%ib = ib
```

```
  rhows%ikbz = ikbz
  rhows%okbz = okbz
  rhows%ob_start_end = (/1, ..., N/)
  rhows%is_ibz = is_ibz
  call isdf_get_rho(rhof, rhows)
```

Listing 1: Example Fortran Code

III. VALIDATION OF THE ISDF ALGORITHM

We use bulk silicon as the test system and neglect spin polarization. In the primitive unit cell, the system contains two silicon atoms, resulting in four occupied states. Calculations are performed using $2 \times 2 \times 2$ and $4 \times 4 \times 4$ **k**-point meshes. For each **k**-point mesh, we compute the Hartree–Fock energies for bands 1 to 15 across all degenerate **k**-point groups. Our results are then compared with those obtained using the original `yambo` code. The comparison is shown in the figure below.

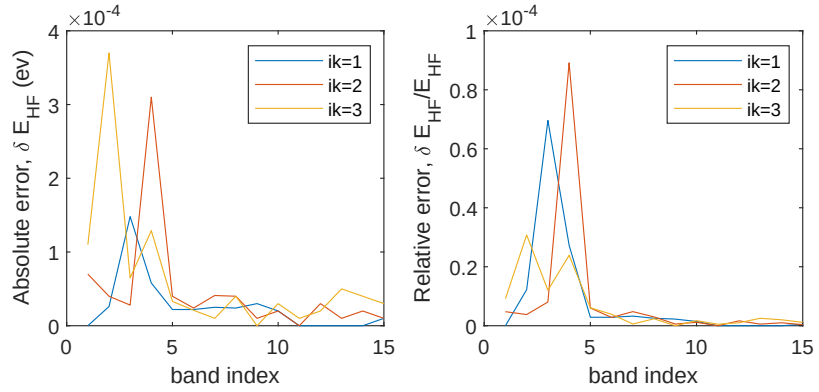

Figure 1: Silicon bulk, use $2 \times 2 \times 2$ **k**-point meshes, $N_\alpha = 379$.

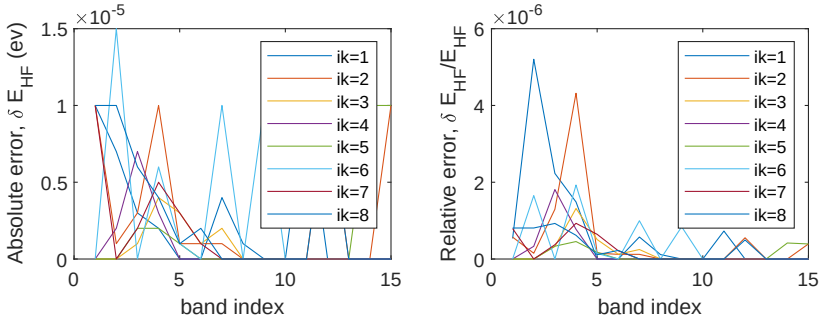

Figure 2: Silicon bulk, use $4 \times 4 \times 4$ **k**-point meshes, $N_\alpha = 1753$.

The above results show that, the ISDF-based density approximation is capable of delivering reliable Hartree-Fock energies for the simpliest tested cases.

A. Fock Energies in Current `Yambo`

In `yambo`, the fock energies is calculated through the following process. First, after discretizing the **k**-points, we identify all possible momentum values $\mathbf{q}$, denoted as $\{\mathbf{q}_1, \ldots, \mathbf{q}_{N_\mathbf{k}}\}$. In fact, these valid $\mathbf{q}$-points coincide with the set of **k**-points. Next, we partition the first Brillouin zone into l non-overlapping regions, each containing exact one $\mathbf{q}$-points, that together form a complete covering. We denote the region contains $\mathbf{q}_i$ as Ω_i. Then the Fock energies is

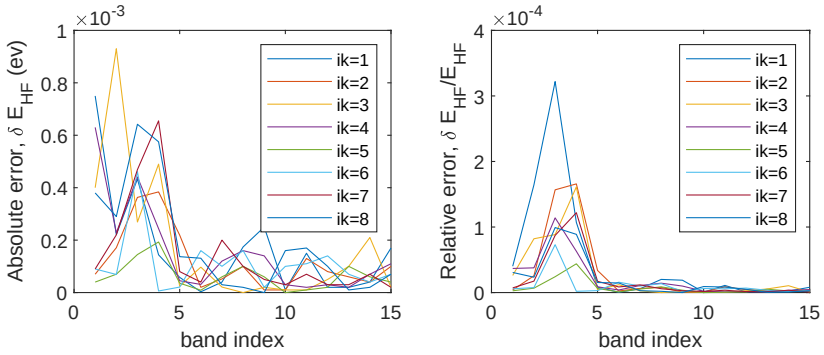

Figure 3: Silicon bulk, use $4 \times 4 \times 4$ **k**-point meshes, $N_\alpha = 876$.

calculated as following

$$\begin{aligned}\Sigma^{\mathrm{F}}_{n\mathbf{k}} &= -\sum_{n_1}\int_{\mathrm{BZ}}\frac{\mathrm{d}\mathbf{q}}{(2\pi)^3} f_{n_1,\mathbf{k}-\mathbf{q}}\sum_{\mathbf{G}} V(\mathbf{q}+\mathbf{G})\overline{\rho_{nn_1}(\mathbf{k},\mathbf{q},\mathbf{G})}\rho_{nn_1}(\mathbf{k},\mathbf{q},\mathbf{G}) \\ &\approx -\sum_{n_1}\sum_{i=1}^{N_{\mathbf{k}}} f_{n_1,\mathbf{k}-\mathbf{q}_i}\sum_{\mathbf{G}}\Big(\int_{\Omega_i}\frac{\mathrm{d}\mathbf{q}}{(2\pi)^3} V(\mathbf{q}+\mathbf{G})\Big)\overline{\rho_{nn_1}(\mathbf{k},\mathbf{q}_i,\mathbf{G})}\rho_{nn_1}(\mathbf{k},\mathbf{q}_i,\mathbf{G}),\end{aligned} \tag{21}$$

and we use $\tilde{V}(\mathbf{q}_i,\mathbf{G})$ to represent the integrale above.

Since `yambo` use Monto–Carlo simulation to calculate $\tilde{V}(\mathbf{q}_i,\mathbf{G})$, making it costy. Therefore, `Yambo` computes the Coulomb potential $\tilde{V}(\mathbf{q}_i,\mathbf{G})$ only over the set of **q**-points that have been reduced by symmetry. We denote the full set of **q**-points as $\mathcal{K}$ and the reduced set as $\mathcal{K}'$.

For any $\mathbf{q}_i \in \mathcal{K}$, there exist a $\tilde{\mathbf{q}}_i \in \mathcal{K}'$ and a point group operator $\hat{R}_i$ satisfies $\hat{R}_i\tilde{\mathbf{q}}_i = \mathbf{q}_i$. Applying this symmetric to $\tilde{V}(\mathbf{q}_i,\mathbf{G})$ leads to

$$\tilde{V}(\mathbf{q}_i,\mathbf{G}) = \tilde{V}(\underline{R}_i\tilde{\mathbf{q}}_i, \underline{R}_i\underline{R}_i^{-1}\mathbf{G}) = \tilde{V}(\tilde{\mathbf{q}}_i, \underline{R}_i{}^{-1}\mathbf{G}). \tag{22}$$

We substitute Eq. (22) into Eq. (21), perform a change of variables $\mathbf{G} = \underline{R}_i\mathbf{G}$, and obtain

$$\Sigma^{\mathrm{F}}_{n\mathbf{k}} \approx -\sum_{n_1}\sum_{i=1}^{N_{\mathbf{k}}} f_{n_1,\mathbf{k}-\mathbf{q}_i}\sum_{\mathbf{G}}\tilde{V}(\tilde{\mathbf{q}}_i,\mathbf{G})\overline{\rho_{nn_1}(\mathbf{k},\mathbf{q}_i,\underline{R}_i\mathbf{G})}\rho_{nn_1}(\mathbf{k},\mathbf{q}_i,\underline{R}_i\mathbf{G}). \tag{23}$$

`Yambo` calculates Eq. (23) in `XCo_Hartree_Fock` using the following procedure

1. **For** n and $\mathbf{k} \in \mathcal{K}'$
2. **For** $\mathbf{q}_i \in \mathcal{K}$
3. find corresponding $\underline{R}_i$ and $\tilde{\mathbf{q}}_i$
4. Calculate $\tilde{V}(\tilde{\mathbf{q}}_i,\mathbf{G})$ in `scatter_Gamp`
5. **For** n_1
6. Calculate $\rho_{nn_1}(\mathbf{k},\mathbf{q}_i,\underline{R}_i\mathbf{G})$ in `scatter_Bamp`
7. Summation over **G**, multiplying desired coefficient, and add it into `QP_Vnl_xc`
8. end **For**
9. end **For**
10. end **For**

The remain problem is how to calculate $\rho_{nn_1}(\mathbf{k},\mathbf{q}_i,\underline{R}_i\mathbf{G})$, which can be expressed using Bloch wavefunctions and $\mathbf{k}$-points in the first Brillouin zone as

$$\begin{aligned}\rho_{nn_1}(\mathbf{k},\mathbf{q},\mathbf{G}) &= \int \mathrm{d}\mathbf{r}\overline{\psi_{n\mathbf{k}}(\mathbf{r})}\psi_{n_1\mathbf{k}-\mathbf{q}}(\mathbf{r})\mathrm{e}^{\mathrm{i}(\mathbf{G}+\mathbf{q})\mathbf{r}} \\ &= \int \mathrm{d}\mathbf{r}\overline{u_{n\mathbf{k}}(\mathbf{r})}u_{n_1\mathbf{k}_1}(\mathbf{r})\mathrm{e}^{\mathrm{i}\mathbf{G}\mathbf{r}}\mathrm{e}^{-\mathrm{i}\mathbf{G}_1\mathbf{r}},\end{aligned} \tag{24}$$

where $\mathbf{k}_1$ is in the first Brillouin zone, $\mathbf{G}_1$ is a reciprocal space lattice, and satisfies $\mathbf{k}_1+\mathbf{G}_1=\mathbf{k}-\mathbf{q}$.

`Yambo` applies symmetric properties to wavefunctions, reducing space cost, which means we only have bloch wavefunctions $u_{n\mathbf{k}}(\mathbf{r})$ with $\mathbf{k}\in\mathcal{K}'$. In the calculation of Hartree–Fock energies Eq. (23), $\mathbf{k}$ is always taken from the symmetry-reduced $\mathbf{k}$-points set $\mathcal{K}'$, whereas $\mathbf{k}_1$ may belong to $\mathcal{K}'$. Suppose a point group operation $\hat{S}$ with representation $\underline{S}$ in reciprocal space, that matches some $\mathbf{k}'\in\mathcal{K}'$ into $\mathbf{k}_1$, i.e., $\underline{S}\mathbf{k}'=\mathbf{k}_1$, then the symmetric of the system suggests

$$u_{n\mathbf{k}_1}(\mathbf{r}) = u_{n\underline{S}\mathbf{k}'}(\mathbf{r}) = u_{n\mathbf{k}'}(\underline{S}^{-1}\mathbf{r}). \tag{25}$$

We now apply Eq. (25) into Eq. (24) to obtain an useful expression of density as

$$\rho_{nn_1}(\mathbf{k},\mathbf{q},\mathbf{G}) = \int \mathrm{d}\mathbf{r}\overline{u_{n\mathbf{k}}(\mathbf{r})}u_{n_1\mathbf{k}'}(\underline{S}^{-1}\mathbf{r})\mathrm{e}^{\mathrm{i}\mathbf{G}\mathbf{r}}\mathrm{e}^{-\mathrm{i}\mathbf{G}_1\mathbf{r}}, \qquad \mathbf{k}_1+\mathbf{G}_1=\mathbf{k}-\mathbf{q}, \qquad \underline{S}\mathbf{k}'=\mathbf{k}_1, \tag{26}$$

and the rotated density $\rho_{nn_1}(\mathbf{k},\mathbf{q}_i,\underline{R}_i\mathbf{G})$ is just replacing $\mathbf{G}$ with rotated $\underline{R}_i\mathbf{G}$ as

$$\rho_{nn_1}(\mathbf{k},\mathbf{q},\underline{R}_i\mathbf{G}) = \int \mathrm{d}\mathbf{r}\overline{u_{n\mathbf{k}}(\mathbf{r})}u_{n_1\mathbf{k}'}(\underline{S}^{-1}\mathbf{r})\mathrm{e}^{\mathrm{i}\underline{R}_i\mathbf{G}-\mathbf{G}_1\mathbf{r}}, \qquad \mathbf{k}_1+\mathbf{G}_1=\mathbf{k}-\mathbf{q}, \qquad \underline{S}\mathbf{k}'=\mathbf{k}_1. \tag{27}$$

To calculate Eq. (27), `Yambo` perform permutations after FFT rather than multiplying the coefficient first then apply FFT. In Eq. (26), we let $\mathbf{G}'=\mathbf{G}-\mathbf{G}_1$, and have

$$\rho_{nn_1}(\mathbf{k},\mathbf{q},\mathbf{G}'+\mathbf{G}_1) = \int \mathrm{d}\mathbf{r}\overline{u_{n\mathbf{k}}(\mathbf{r})}u_{n_1\mathbf{k}'}(\underline{S}^{-1}\mathbf{r})\mathrm{e}^{\mathrm{i}\mathbf{G}'\mathbf{r}} = \mathtt{iFFT}\Big(\overline{u_{n\mathbf{k}}(\cdot)}u_{n_1\mathbf{k}'}\big(\underline{S}^{-1}(\cdot)\big)\Big)(\mathbf{G}'), \tag{28a}$$

$$\Rightarrow \rho_{nn_1}(\mathbf{k},\mathbf{q},\underline{R}_i\mathbf{G}) = \mathtt{iFFT}\Big(\overline{u_{n\mathbf{k}}(\cdot)}u_{n_1\mathbf{k}'}\big(\underline{S}^{-1}(\cdot)\big)\Big)(\underline{R}_i\mathbf{G}-\mathbf{G}_1), \qquad \mathbf{k}_1+\mathbf{G}_1=\mathbf{k}-\mathbf{q}, \qquad \underline{S}\mathbf{k}'=\mathbf{k}_1, \tag{28b}$$

where Eq. (28b) is exact the implementation in `yambo`.

`Yambo` calculate Eq. (27) in `scatter_Bamp_*` using the following procedure

1. Read $u_{n\mathbf{k}}(\mathbf{r})$ and $u_{n_1\mathbf{k}'}(\underline{S}^{-1}\mathbf{r})$ using `wf_apply_symm`. The indices mapping from $\mathbf{r}$ to $\underline{S}^{-1}\mathbf{r}$ is provided in `fft_rot_r`
2. Calculate $\rho(\mathbf{r})=\overline{u_{n\mathbf{k}}(\mathbf{r})}u_{n_1\mathbf{k}'}(\underline{S}^{-1}\mathbf{r})$
3. Apply FFT to $\rho(\mathbf{r})$ to get $\rho_{\mathbf{t}}(\mathbf{G})$.
4. Apply $\mathbf{G}$-indices permutation to $\rho_{\mathbf{t}}$, first with respect to the rotation $\underline{R}_i(\cdot)$, then with respect to the translation $\cdot-\mathbf{G}_1$. The former one is provided in `g_rot`, and the latter one is in `fft_g_table`

B. ISDF in `Yambo` Hartree–Fock

We need to modify our ISDF algorithm to adapt the structure and require of `Yambo`. Here we write our implementation.

First, density used in Hartree–Fock involves a single $\mathbf{k}$-point index and a momentum index $\mathbf{q}$, rather than two $\mathbf{k}$-point indices. The relationship is straightforward as $\mathbf{p}=\mathbf{k}-\mathbf{q}+\mathbf{G}_0$, and we can perform the following correspondence in advance

$$\rho_{nm}(\mathbf{k},\mathbf{q},\mathbf{G}) \triangleq \sum_{\mathbf{r}}\overline{u_{n\mathbf{k}}(\mathbf{r})}u_{m\mathbf{p}}(\mathbf{r})\mathbf{e}^{\mathbf{i}\mathbf{G}_0\cdot\mathbf{r}} = \rho_{n\mathbf{k}m\mathbf{p}}(\mathbf{G}), \qquad \mathbf{p}=\mathbf{k}-\mathbf{q}+\mathbf{G}_0, \tag{29}$$

and use this density to continue the process provided in Eq. (3). Second, we need to figure out the symmetric impact on ρ. Thanks to `WF_apply_symm`, we can read $u_{m\mathbf{p}}(\mathbf{r})=u_{m\mathbf{k}'}(\underline{S}^{-1}\mathbf{r})$ with $\underline{S}\mathbf{k}'=\mathbf{k}$ without difficulty.

Now we get all we need for ISDF algorithm, but there are still one problem. `Yambo` use rotated density in Eq. (23). Why it is the case? Can we rotate $\tilde{V}$ with $\underline{R}_i^{-1}$?

IV. APPROXIMATE FOUR-CENTER INTEGRAL INSTEAD OF DENSITY

In the code, we prefer to approximate four-center integral

$$\langle n\mathbf{k}m\mathbf{p}|\hat{V}(\mathbf{q})|n\mathbf{k}m\mathbf{p}\rangle = \sum_{\mathbf{G}} \overline{\rho_{n\mathbf{k}m\mathbf{p}}(\mathbf{G})} V(\mathbf{q};\mathbf{G})\rho_{n\mathbf{k}m\mathbf{p}}(\mathbf{G}) \tag{30}$$

rather than approximate the density $\rho_{n\mathbf{k}m\mathbf{p}}(\mathbf{G})$. The reason is computational cost.

We take the Hartree–Fock calculation to illustrate this concept. For convenience, we refer to the original method (current `yambo`), the method that approximates the four-center integrals, and the method that approximates the density as method 1, method 2, and method 3, respectively.

`Yambo` calculates Eq. (23) in `XCo_Hartree_Fock` using the following procedure

1. **For** n and $\mathbf{k} \in \mathcal{K}'$
2. **For** $\mathbf{q}_i \in \mathcal{K}$
3. find corresponding $\underline{R}_i$ and $\tilde{\mathbf{q}}_i$
4. Calculate $\tilde{V}(\tilde{\mathbf{q}}_i, \mathbf{G})$ in `scatter_Gamp`
5. **For** n_1 is valence band
6. Calculate $\rho_{nn_1}(\mathbf{k}, \mathbf{q}_i, \underline{R}_i\mathbf{G})$ in `scatter_Bamp`
7. Calculate $\langle n\mathbf{k}m\mathbf{p}|\hat{V}(\mathbf{q})|n\mathbf{k}m\mathbf{p}\rangle$, and add it into $\Sigma^n_{\mathbf{HF}}$
8. end **For**
9. end **For**
10. end **For**

method 2 only changes step 6, while method 3 changes step 6 and 7.

We focus on these two step and figure out the calculation cost. Suppose that

- N_{qp}: Number of required $(n, \mathbf{k})$-pair.
- $N_{\mathbf{k}}$, $N'_{\mathbf{k}}$: Number of original $\mathbf{k}$-points set and irreducible one.
- N_v: Number of valence band.
- N_α: Number of ISDF helper functions and interpolative points.
- $N_{\mathbf{G}}$: Number of grids in reciprocal space.

Method 1 takes $O(N_{\mathbf{G}} \log N_{\mathbf{G}})$ loops in each step 6 and step 7. The main computational load lies in the two FFT operations in step 6. Method 2 need `gemv` to calculate the density, causing $O(N_\alpha N_{\mathbf{G}})$ in each step 6, which indeed is more expensive than Method 1. Method 3 do the following calculation to replace inner `For` loops.

5. Calculate $\Upsilon^{\mathbf{q}}_{\alpha,\beta} = \sum_{\mathbf{G}} \overline{p_\alpha(\mathbf{G})}\tilde{V}(\tilde{\mathbf{q}}_i, \mathbf{G})p_\beta(\mathbf{G})$, and if $\Upsilon^{\mathbf{q}}$ was calculated before, load it.
6. **For** n_1 is valence band
7. Calculate

$$\langle n\mathbf{k}m\mathbf{p}|\hat{V}(\mathbf{q})|n\mathbf{k}m\mathbf{p}\rangle = \sum_{\alpha,\beta} \psi_{n\mathbf{k}}(\mathbf{x}_\alpha)\overline{\psi_{m\mathbf{k}}(\mathbf{x}_\alpha)}\Upsilon_{\alpha,\beta}\overline{\psi_{n\mathbf{k}}(\mathbf{x}_\beta)}\psi_{m\mathbf{k}}(\mathbf{x}_\beta),$$

and add it into $\Sigma^n_{\mathbf{HF}}$

8. end **For**

In method 3, step 5 costs $2N_\alpha^2 N_{\mathbf{G}}$, which dominates step 7 since $N_{\mathbf{G}}$ dominates N_α.

We list all computational cost in the following table

In a nutshell, if ISDF is applied only at the `bamp` level, the computational cost in Hartree–Fock calculations will significantly increase compared to the direct method. In contrast, applying ISDF at the four-center integral level substantially reduces the computational cost compared to using it at the `bamp` level.

	Method 1	Method 2	Method 3
Step 5	$O(N_{qp}N_vN_{\mathbf{G}}\log N_{\mathbf{G}})$	$O(N_{qp}N_vN_{\mathbf{G}}N_\alpha)$	$O(N'_{\mathbf{k}}N_{\mathbf{G}}N_\alpha^2)$
Step 6	/	/	/

V. SYMMETRIC WITH RESPECT TO WAVEFUNCTIONS

In this section, we briefly introduce the symmetricity of systems. We only provide several definitions, notations, claims, propositions, etc.

A. Space group in real space

1. Definition 1[Space group of a system]: We say that a three-dimensional isometric transformation group $\mathcal{G}$ is the **space group of a system** if its elements map the crystal onto itself.
2. Notation 1: We use R_α to denote a point group operation, where the point group operation is a set of **isometric transformations that leave a certain point fixed**.
3. Notation 2: We use $\{R_\alpha|\tau\}$ to denote an element of $\mathcal{G}$, where R_α denotes the point group operation and τ denotes translation operations.
4. Theorem 1[Representation of $\mathcal{G}$] The matrix
$$\begin{bmatrix} 1 & 0 \\ \tau & \alpha \end{bmatrix}$$
forms a representation for the space group operator $\{R_\alpha|\tau\}$
5. Theorem 2[Translation subgroup] All the elements of the space group $\mathcal{G}$ that are of the form $\{I_3|\tau\}$ form a translation group $\mathcal{T}$, which is a subgroup of $\mathcal{G}$.
6. Definition 2: All the translation vector of translation operator forms *Bravais lattice*
7. Definition 3[Primitive vectors and direct lattice]: The primitive vectors refer to the generator of Bravais lattice, denoted by $(\boldsymbol{a}_1, \boldsymbol{a}_2, \boldsymbol{a}_3)$, and the direct lattice refers to a vector located within the primitive lattice or on the boundary near the origin.
8. Notation 3: We denote C_α as the right coset of $\{R_\alpha|\tau\}$ with respect to $\mathcal{T}$, i.e.,
$$C_\alpha = \left\{\{I_3|\tau'\}\{R_\alpha|\tau\} \mid \{I_3|\tau'\} \in \mathcal{T}\right\}$$
9. Theorem 3: The right cosets C_α form a factor group of $\mathcal{G}$.
10. We can factor $\{R_\alpha|\tau\} = \{R_\alpha|\tau_\alpha\}\{I_3|\mathbf{R}_\tau\}$, with $\mathbf{R}_\tau$ as an element of Bravais lattice, and τ_α is inside the direct lattice.
11. Definition 4 [Symmorphic group and nonsymmorphic group]: If there exist a suitable choice of origin in the direct lattice
12. Notes: factor group –> we only care about the situation with translation vector "inside" one lattice.

B. Space group in Reciprocal Space

We hope to adopt $\mathcal{G}$ into wavefunctions to generarte some symmetric. We based on $\{R_\alpha|\tau\}$ to construct $\hat{P}_{\{R_\alpha|\tau\}}$, which maps functions into functions, that leaves the periodic potential $V(\mathbf{r})$ invariant when τ is a Bravais lattice.

- Definition 5 [Reciprocal lattice vectors]: We use $\mathbf{R}$ to denote **Bravais lattice**, and $\mathbf{G}$ to denote **reciprocal lattice vector**.
- Notation 4: We use $(\boldsymbol{a}_1, \boldsymbol{a}_2, \boldsymbol{a}_3)$ and $(\boldsymbol{b}_1, \boldsymbol{b}_2, \boldsymbol{b}_3)$ to denote the primitive translation vector and the primitive reciprocal lattice vector, respectively.

- (First apply translation group $\mathcal{T}$) We defined that

$$\hat{P}_{\{I_3|\tau\}}\psi(\mathbf{r}) = \psi(\mathbf{r}+\tau).$$

  One can show this is well-defined and consistency with respect to $V(\mathbf{r})$, i.e., the result after translation still satisfies Schrodinger equation.

- Notation 5: For a translation vector τ, we write $\tau = \sum_{i=1}^{3} n_i \boldsymbol{a}_i = \sum_{i=1}^{3} \tau_i$. One can verify the trivial factorization

$$\{I_3|\tau\} = \{I_3|\tau_1\}\{I_3|\tau_2\}\{I_3|\tau_3\},$$

  and the operators on the r.h.s. are communitive.

- Theorem 4 [Representation of $\mathcal{T}$]: Suppose N_i is the number of unit cell along $\boldsymbol{a}_i$, then the irreducible representation of the translation group $\mathcal{T}$ is

$$\left[\mathrm{diag}(\mathrm{e}^{\mathrm{i}k_1 n_1 a_1}, \mathrm{e}^{\mathrm{i}k_2 n_2 a_2}, \mathrm{e}^{\mathrm{i}k_3 n_3 a_3})\right]$$

  Here $k_i = 2\pi m_i / L_i$, where $m_i \in \{0, 1, \cdots, N_i - 1\}$, and $L_i = N_i \|\boldsymbol{a}_i\|_2$.

- (Then reduce translation group $\mathcal{T}$ from all translation operators) Theorem 5[Bloch's theorem] If a wavefunction $\psi_{\mathbf{k}}$ satisfied

$$\hat{P}_{\{I_3|\mathbf{R}\}}\psi_{\mathbf{k}}(\mathbf{r}) = \mathrm{e}^{\mathrm{i}\mathbf{k}\mathbf{R}}\psi_{\mathbf{k}}(\mathbf{r})$$

  for all possible $\mathbf{R}$ as Bravais lattice, then for any τ, we have

$$\hat{P}_{\{I_3|\tau\}}\psi_{\mathbf{k}}(\mathbf{r}) = \psi_{\mathbf{k}}(\mathbf{r}+\tau),$$

  and

$$\hat{P}_{\{I_3|\tau\}}\psi_{\mathbf{k}}(\mathbf{r}) = \mathrm{e}^{\mathrm{i}\mathbf{k}\tau}\psi_{\mathbf{k}}(\mathbf{r}).$$

  .. Note: There must be some mistakes here, or typos from the textbook ... The prove is not consistent, as far as I suppose.

- Definition 6 [Bloch wavefunction]: $\psi_{\mathbf{k}}(\mathbf{r}) = \mathrm{e}^{\mathrm{i}\mathbf{k}\mathbf{r}} u_{\mathbf{k}}(\mathbf{r})$.

We define the translation operators on wavefunctions, from which we introduce **k**-points in reciprocal space. Now, we need to defined the following things

- Apply space operator to reciprocal space,
  - Since $\mathbf{k}$ is designed to be invariant under translation, i.e., $\{I_3|\tau\}\mathbf{k} = \mathbf{k}$ for all τ, we only consider point group operation.
  - Intuition: We know the point group operation in real space, we hope the operation in reciprocal space maintains orthogonality relation, i.e.,

$$\hat{P}_\alpha \mathbf{R}_m \cdot \hat{P}_\alpha \mathbf{G}_m = \mathbf{R}_m \cdot \mathbf{G}_m = 2\pi l, \qquad l \in \mathbb{Z},$$

    where $\mathbf{R}_m$ is a Bravais lattice in real space, and $\mathbf{G}$ is that in reciprocal space.
  - One can shows that, if we let $\hat{P}_\alpha \mathbf{K}_m = (\{R_\alpha|\tau\})^{-1}\mathbf{K}_m$ (R.h.s refers to first regard $\mathbf{K}_m$ as a vector in real space, then apply the operator), orthogonality relation is maintained.
  - We extend the definition on lattice to reciprocal space as ...
  - Definition X [Space group operation on reciprocal space]: For a space operator $\{R_\alpha|\tau\} \in \mathcal{G}$, we define its operation on reciprocal space as

$$\{R_\alpha|\tau\}\mathbf{k} = \{R_\alpha|0\}\mathbf{k} = (\{R_\alpha|\tau\})^{-1}\mathbf{k},$$

    where the last equation refers to first regard $\mathbf{k}$ as a vector in real space, then apply the operation.
  - Definition 6 [Equivalent $\mathbf{k}$ points] If $\mathbf{k}' = \mathbf{k} + \mathbf{G}$ for some GG in reciprocal lattice, we say that $\mathbf{k}'$ and $\mathbf{k}$ are equivalent $\mathbf{k}$ points.

- Definition 7 [Group of the wave vector] For a given $\mathbf{k}$, the group of the wave vector is formed by the set of space group operations, which transform $\mathbf{k}$ into itself, or with an extra shift of reciprocal lattice.
- Definition 8 [Star of $\mathbf{k}$] For a given $\mathbf{k}$, the star of $\mathbf{k}$ is a set of $\mathbf{k}$ points in reciprocal space, formed by apply the space group to $\mathbf{k}$ and select all inequivalent $\mathbf{k}$ points.
- Note: |Group of the wave vector| $\times$ |Star of $\mathbf{k}$| $= |\mathcal{G}/\mathcal{T}|$

• Apply point group operator to wavefunctions.

- (We know how to apply translation operator on wavefunctions, Now try point group operation)
- Definition 9: For a wavefunction $\psi_{\mathbf{k}}(\mathbf{r})$ (remind how we define $\psi_{\mathbf{k}}$), we define

$$\hat{P}_{\{R_\alpha|0\}}\psi_{\mathbf{k}}(\mathbf{r}) = \psi_{R_\alpha\mathbf{k}}(\mathbf{r}).$$

  One can show that, first $\psi_{R_\alpha\mathbf{k}}(\mathbf{r})$ is a wavefunction satisfies Schrodinger equation, second $\psi_{R_\alpha\mathbf{k}}(\mathbf{r})$ satisfies $R_\alpha\mathbf{k}$ under translation operations.
- Due to the uniqueness of the solution to the equation, we know that the wavefunction obtained in this way is identical to the one represented by a wavefunction at $R_\alpha\mathbf{k}$, which would be obtained by directly solving for $\psi(\mathbf{r})$ under the corresponding translational transformation.
- For a space group operation $\hat{P}_{\{R_\alpha|\tau\}}$, we have

$$\hat{P}_{\{R_\alpha|\tau\}}\psi_{\mathbf{k}}(\mathbf{r}) = \mathrm{e}^{\mathrm{i}R_\alpha\mathbf{k}\tau}\psi_{R_\alpha\mathbf{k}}(\mathbf{r}).$$